NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Mestrado em Engenharia Eletrotécnica e de Computadores

Computer Vision

A Motion-Based Interactive Game Using Computer Vision and GPU Acceleration

---

Authors:

Filipe Cavalheiro (70119) fs.cavalheiro@campus.fct.unl.pt
Diogo Novais (70118) dm.novais@campus.fct.unl.pt

2025/2026 – 1st Semester, DEEC

Contents

List of Figures

1 Introduction

This project implements an interactive motion-based game that leverages advanced computer vision techniques to create an engaging user experience. The system combines face detection, pose estimation, template matching, and GPU-accelerated image processing to enable players to control game elements using body movements captured through a standard webcam.

The project demonstrates the practical application of multiple computer vision algorithms, including deep neural networks (DNN) for detection tasks and custom OpenCL kernels for high-performance image processing operations. The game implements a "Simon Says" style interaction where players must position their hands over colored squares based on text prompts, creating a challenging exercise in both visual perception and physical coordination.

A video demonstatios of the project can be seen at: TAPDI - Apresentação do projeto

1.1 Project Objectives

For this project we were tasked with using real time template matching using the GPU and kernels in order to track and stabelize/center the games user in the center of the screen at all times, at the same time pose estimation was done via the OpenPose library having a ROI for analysis based on the location of the users face, detected via a DNN. Finally a game of our choice, in this case "Simon Says", must be implement with this technology in mind.

2 Technical Architecture

2.1 System Overview

The system architecture consists of several interconnected modules, each responsible for specific computer vision tasks. Figure 1 illustrates the overall data flow through the system.

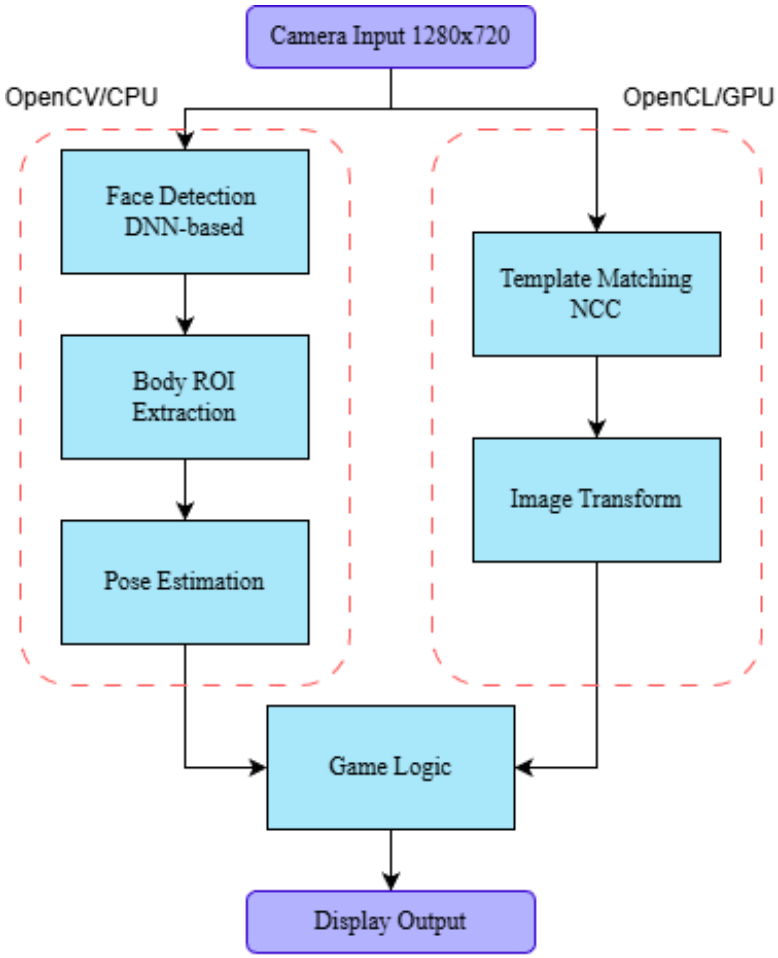

Figure 1: System architecture and data flow diagram

2.2 Module Descriptions

The multiple modules shown in fig. 1 have are below described:

- Face Detection DNN-based (section 3.1)
- Template Matching and Image Transformation are both described in (section 3.2)
- Body ROI Extraction (section 3.3)
- Pose Estimation (section 3.4)
- Finally the game logic is applied as shown in (section 4)

3 In Depth Module Descriptions

The system implements face detection using a TensorFlow-based deep neural network for later use by OpenPose as well as template matching. The use of a DNN instead of a traditional

Haar cascade classifiers is that Tthis approach provides superior accuracy and robustness to varying lighting conditions and face orientations. As has been shown and proven during classes along the semester, more precissely in TP9-10.

3.1 DNN-Based Detection Pipeline

The detection process involves several key steps:

1. **Blob creation:** The input frame is preprocessed into a blob with fixed dimensions (800x800)
2. **Forward pass:** The blob is fed through the neural network
3. **Confidence filtering:** Detections with confidence scores above 0.7 are retained
4. **Coordinate scaling:** Bounding box coordinates are scaled back to original image dimensions

3.2 Template matching: OpenCL Implementation

The project leverages GPU acceleration through OpenCL for computationally intensive operations. Three primary kernels are implemented in C and compiled at runtime:

3.2.1 Template Average Subtraction

This kernel prepares the face template for normalized cross-correlation by subtracting the mean intensity from each pixel. The operation is performed in parallel across all pixels, with each GPU thread processing one pixel location. The kernel reads from an input image, subtracts a pre-computed mean value (stored as a 4-component vector for RGBA channels), and writes the result to an output image. Boundary conditions are handled by checking if coordinates exceed image dimensions. Subtraction of the image average is done in seperate since the template average subtraction only need to be done once while the rest of the kernels are used multiple times.

3.2.2 Normalized Cross-Correlation (NCC)

The NCC kernel implements efficient template matching using local memory tiling for improved memory access patterns. The implementation uses work groups of 8x8 threads, with the template dimensions fixed at 105x90 pixels.

The kernel operates in three phases:

Phase 1: Cooperative Tile Loading

Each work group cooperatively loads a tile of the input image into fast local (shared) memory. The tile size is larger than the work group size to accommodate the template dimensions plus overlap. Multiple threads within each work group participate in loading different portions of the tile, ensuring efficient memory access patterns. Since the stride is 1 by havin a local memory of the size of the template plus 8 pixel per side making it possible for each workgroup to handle a total of 64 process at once. The decision to use a work group of 8x8 is due to the size limit of the workgroup memory, passing to 16x16 would not be feasable due to hardware constrants

as indicated section A.

Phase 2: Statistical Computation
After synchronization, each thread computes statistics for its corresponding sub-image region. This includes calculating the mean intensity and sum of squared intensities across the template-sized window. These values are computed by iterating through the local memory.

Phase 3: Correlation Calculation
The final correlation value is computed by multiplying corresponding pixels from the sub-image (minus its mean) with the pre-processed template (already mean-subtracted). The result is normalized by the geometric mean of the sum of squares, producing a correlation coefficient between -1 and 1. This value is then scaled to the range [0, 255] for output as an 8-bit image.

For a better understanding the NCC formula implemented is shown below:

$$\rho(x,y) = \frac{\sum_{i,j}(T(i,j) - \bar{T})(I(x+i,y+j) - \bar{I}_{x,y})}{\sqrt{\sum_{i,j}\left[T(i,j)^2\right] \cdot \sum_{i,j}\left[I(x+i,y+j)^2\right]}} \tag{1}$$

$$\bar{T} = \frac{1}{w \cdot h} \cdot \sum_{i,j} T(i,j) \quad \bar{I}_{x,y} = \frac{1}{w \cdot h} \cdot \sum_{i,j} I(i+x, j+y)$$

where T is the template, I is the input image, and $\bar{T}$, $\bar{I}_{x,y}$ are their respective means.

3.2.3 Linear Image Transformation

This kernel performs image translation to center the detected face in the frame. Each thread processes one output pixel by reading from a shifted location in the input image. The kernel parameters include the image dimensions and the x/y shift amounts. Boundary checking ensures that reads from shifted positions remain within valid image coordinates. Pixels that would map outside the input image bounds are skipped, effectively creating black borders where the shifted image doesn't provide data. The sampler is configured with edge clamping to handle boundary conditions gracefully.

3.2.4 Used template

The template used for template matching is the one shown in fig. 2

Figure 2: Face Template used for the project

The choice of this face template and explation of its low resolution come from the fact that despite the template matching only caring about the face, open pose need a full body reference in order to work optimally making so that the subject is far away from the camera and redusing the dimension/quality of the face template being the same only 105x90 pixels.

3.3 Body Region of Interest Extraction

Once a face is detected, the system extracts a body region of interest (ROI) for pose estimation. The ROI is calculated based on anthropometric proportions relative to the detected face dimensions.

The ROI calculation uses empirically determined scaling factors:

- Horizontal extent: 6x face width on each side of face center
- Vertical extent: From 40 pixels above face to bottom of frame

3.4 Pose Estimation

Pose estimation is performed using the OpenPose MPI model, which detects 15 body keypoints, including the head, shoulders, elbows, wrists, hips, knees, and ankles. More specifically, the model *pose_deploy_linevec_faster_4_stages* with weights *pose_iter_160000.caffemodel* is used. Despite the model's capability to detect the full body, only keypoints 4 and 7 (corresponding to the right and left hands) are utilized in this work.

3.4.1 OpenPose Implementation

Before invoking the OpenPose inference, the input image is downscaled to a resolution of 368×368 pixels, which is the recommended input size for pose estimation with this model.

The pose estimation process consists of the following stages:

1. Conversion of the input image into a normalized blob (pixel values divided by 255)
2. Generation of confidence heatmaps for each body part
3. Extraction of maximum-confidence locations from each heatmap
4. Filtering of keypoints based on a confidence threshold of 0.05
5. Scaling of the extracted coordinates back to the original image dimensions

Due to the high computational cost of pose estimation, this process is executed only once every 10 frames, and the body keypoint positions are updated accordingly. This significantly improves runtime performance, as running OpenPose on every frame yields approximately 2 frames per second, which is inline with the performance expectations documented by OpenPose, fig. 3 [1].

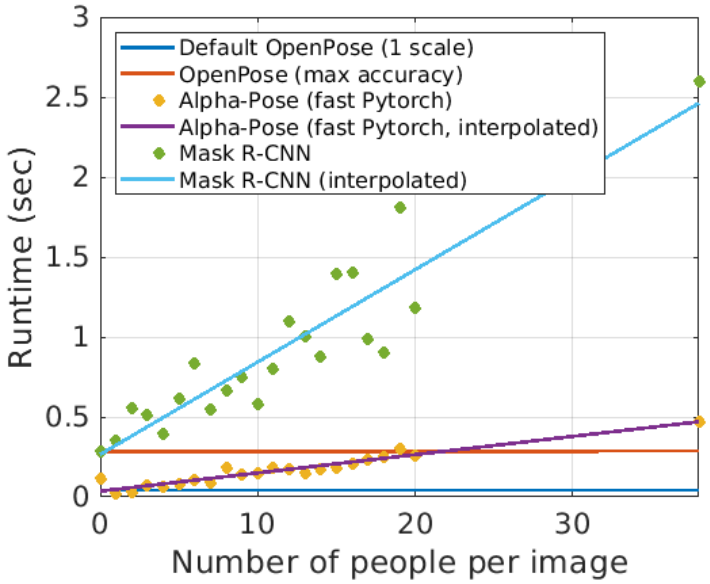

Figure 3: OpenPose run time for multiple models and numbers of people

3.5 System Pipeline

As a recap of the processes used a step by step process of the system is written bellow:

The complete template matching/GPU pipeline combines multiple processing steps:

1. **Template preprocessing:** Load face template and compute mean-subtracted version
2. **Frame processing:** For each input frame, compute NCC at all positions
3. **Maximum detection:** Find location of maximum correlation (face center)
4. **Centering calculation:** Compute translation offset to center face at frame position (width/2, height/5)
5. **Image transformation:** Apply translation to center the user's face

The pose estimation/CPU pipeline is below:

1. **Blob creation:** The input frame is preprocessed into a blob with fixed dimensions (800x800) and normalized using mean subtraction with values [104, 117, 123]
2. **Forward pass:** The blob is fed through the neural network
3. **Confidence filtering:** Detections with confidence scores above 0.7 are retained
4. **Coordinate scaling:** Bounding box coordinates are scaled back to original image dimensions
5. **ROI is created:** Based on the bouding box a ROI is created.

6. **ROI is downscaled:** ROI is downscaled to 368x368.

7. **OpenPose is ran:** The output points of open pose are stored for use in the game.

With the complete porcessing logic for pose estimation and centering of the player the following section focuses on the game development and implementation.

4 Game Logic and Interaction

4.1 Game States

The game operates in multiple states managed through the main loop:

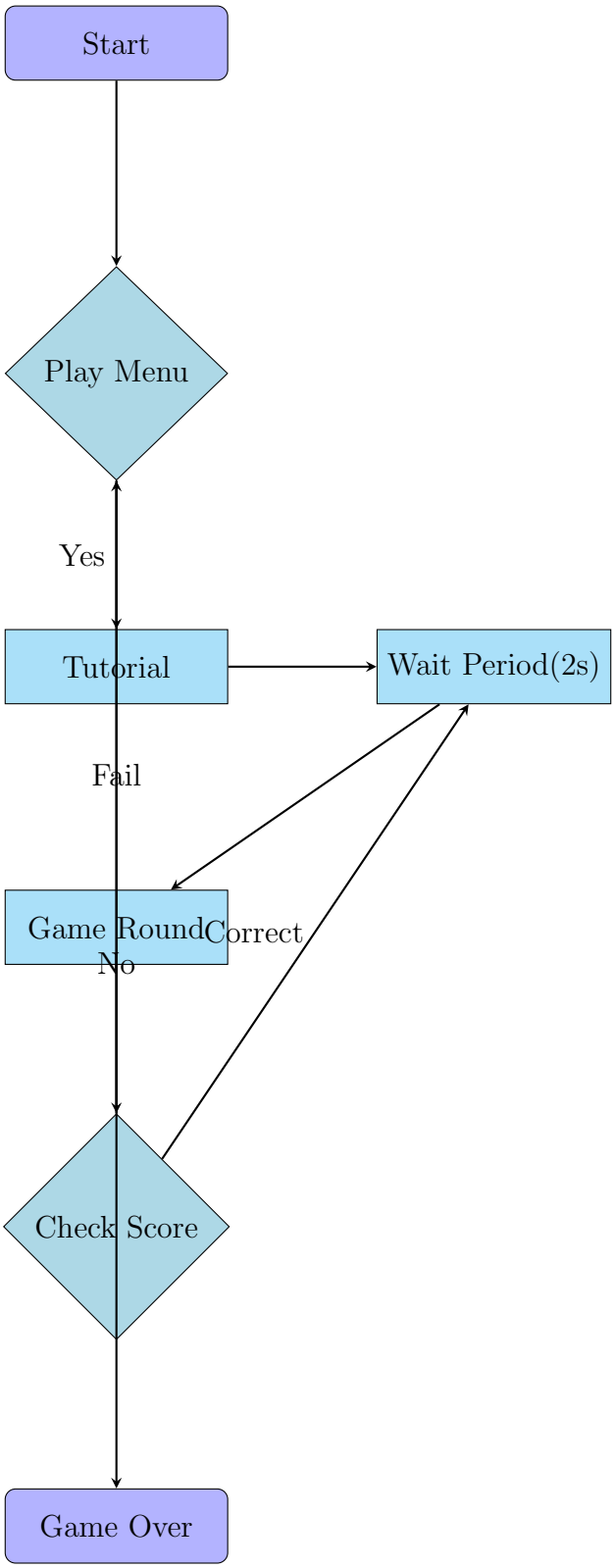

Figure 4: Game state machine

4.1.1 Menu State

In the menu state, the system displays "Yes" and "No" boxes. Players hover their hand over:

- **Yes box** (green, left side): Starts the game
- **No box** (red, right side): Exits the application

4.1.2 Tutorial State

The tutorial displays instructions for 5 seconds explaining the game mechanics before transitioning to the first round.

4.1.3 Game State

During active gameplay:

- Four colored boxes appear: Blue (top-left), Green (top-right), Red (bottom-left), Yellow (bottom-right)
- A color word is displayed in random color at body center
- Player must hover hand over box matching the word (not the text color)
- Time limit starts at 15 seconds and decreases by 10% each round (time decay factor 0.9)
- Score increases on correct selection
- Game resets to menu on: timeout, wrong selection, or multiple hands detected

4.2 Collision Detection

The game uses point-in-rectangle collision detection for hand-box interactions. The system first extracts hand keypoints from the pose estimation results by filtering indices 4 through 13 at intervals of 3, which correspond to wrist positions for both hands. These keypoints are then scaled from the 368×368 crop dimensions back to the original body ROI coordinates.

For each colored box, the system counts how many valid hand keypoints fall within its boundaries. The collision logic uses simple coordinate comparisons: a point is considered inside a box if its x-coordinate falls between the box's left and right edges, and its y-coordinate falls between the top and bottom edges.

The game implements strict matching rules using pattern matching:

- Keypoint coordinates are scaled from 368x368 crop back to original body ROI dimensions
- Only valid (non-None) keypoints are considered
- Exactly one hand in correct box = success
- One hand in wrong box = failure
- Multiple hands detected = failure (prevents cheating)

4.2.1 Text Color Probability Decay

At the beginning of the game, the color of the displayed text is highly likely to match the semantic meaning of the word, making early rounds easier for the player. As the game progresses and the score increases, this probability decreases, increasing the difficulty by encouraging Stroop-effect interference.

The probability of the text color matching the word color is defined as:

$$P_{\text{match}} = \frac{10}{\log_{100}(\text{score} + 1)} \tag{2}$$

At each round, a uniformly distributed random integer $r \in [0, 100]$ is generated. If:

$$r \leq P_{\text{match}} \tag{3}$$

then the text color matches the word. Otherwise, the text color is randomly selected from the remaining colors, excluding the correct one.

This logarithmic decay ensures:

- Near-certain color-word matching at low scores
- Gradual reduction of matching probability as score increases
- A smooth difficulty ramp without abrupt changes

This mechanism maintains early accessibility while progressively increasing cognitive load as the player advances through successive rounds.

References

[1] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "Openpose: Realtime multi-person 2d pose estimation using part affinity fields," 2019. [Online]. Available: https://arxiv.org/abs/1812.08008

A Computer Specification

Filipe Cavalheiro	
Property	**Value**
Name	NVIDIA CUDA
Vendor	NVIDIA Corporation
Version	OpenCL 3.0 CUDA 13.1.86
Device Name	NVIDIA GeForce RTX 2060
Max Compute Units	30
Max Constant Args	9
Max Work Item Dimensions	3
Max Work Item Sizes	[1024, 1024, 64]
Max Work Group Size	1024
Image Support	1
Image2D Max Width	32768
Image2D Max Height	32768
Local Mem Size	49152

Diogo Novais	
Property	**Value**
Name	NVIDIA CUDA
Vendor	NVIDIA Corporation
Version	OpenCL 3.0 CUDA 13.1.86
Device Name	NVIDIA GeForce RTX 5060 Laptop GPU
Max Compute Units	26
Max Constant Args	9
Max Work Item Dimensions	3
Max Work Item Sizes	[1024, 1024, 64]
Max Work Group Size	1024
Image Support	1
Image2D Max Width	32768
Image2D Max Height	32768
Local Mem Size	49152
Preferred Work Group Size Multiple	32

B Developed Kernels

```
import cv2 as cv
import numpy as np
import pyopencl as cl
import math
import matplotlib.pyplot as plt
```

```
from gameManager import GameManager

def subtract_template_avg(gm: GameManager, verbose: bool = False):
    face_template = cv.imread(gm.face_template_location)
    face_template_rgba = cv.cvtColor(face_template, cv.
       COLOR_BGR2RGBA)

    h, w, c = face_template_rgba.shape
    face_template_rgba = face_template_rgba.astype(np.float32)

    # Pre-calculate template averages
    split_img = cv.split(face_template_rgba)
    template_pre_calc = np.array([np.sum(x) / (w * h) for x in
       split_img])

    try:
        img_format = cl.ImageFormat(cl.channel_order.RGBA, cl.
           channel_type.FLOAT)

        # Create OpenCL images
        imageIn = cl.Image(gm.ctx, cl.mem_flags.READ_ONLY | cl.
           mem_flags.COPY_HOST_PTR, img_format, shape=(w, h),
           hostbuf=face_template_rgba)
        imageOut = cl.Image(gm.ctx, cl.mem_flags.WRITE_ONLY,
           img_format, shape=(w, h))

        # Set kernel arguments
        kernel = gm.kernel_subtract_val_to_img
        kernel.set_arg(0, np.int32(w))
        kernel.set_arg(1, np.int32(h))
        kernel.set_arg(2, template_pre_calc)
        kernel.set_arg(3, imageIn)
        kernel.set_arg(4, imageOut)

        # Enqueue kernel execution
        cl.enqueue_nd_range_kernel(gm.commQ, kernel, (w, h), None)
        gm.commQ.finish()

        # Allocate host array to copy result
        output = np.zeros((h, w, 4), dtype=np.float32)
        cl.enqueue_copy(gm.commQ, output, imageOut, origin=(0, 0,
           0), region=(w, h, 1))
        gm.commQ.finish()

        if verbose:
```

```
            img_out = output.astype(np.uint8)
            img_out_rgb = cv.cvtColor(img_out, cv.COLOR_RGBA2BGR)

            plt.imshow(img_out_rgb)
            plt.axis('off')
            plt.show()

            img_out_gray = cv.cvtColor(img_out, cv.COLOR_RGBA2GRAY)
            cv.imshow("frame", img_out_gray)
            cv.waitKey(0)

        face_template_gray = cv.cvtColor(face_template, cv.
           COLOR_RGBA2GRAY)
        sum_square_template = np.sum(np.square(np.array(
           face_template_gray).flatten()))

        return output, sum_square_template

    except Exception as e:
        print("From subtract_template_avg error:", e)

def normalized_correlation_coefficient(input_image,
   template_minus_avg, template_sum_mean, gm: GameManager, verbose:
    bool = False):

    template_minus_avg = cv.cvtColor(template_minus_avg, cv.
       COLOR_RGBA2GRAY)
    input_image = cv.cvtColor(input_image, cv.COLOR_BGR2GRAY)

    #input_image = cv.resize(input_image, (640, 480))
    temp_h, temp_w = template_minus_avg.shape
    img_h, img_w = input_image.shape

    #images are gray scale
    img_format = cl.ImageFormat(cl.channel_order.R, cl.channel_type
       .FLOAT)
    img_format_uint8 = cl.ImageFormat(cl.channel_order.R, cl.
       channel_type.UNSIGNED_INT8)

    # Upload template
    imageIn_template = cl.create_image(
        gm.ctx,
        cl.mem_flags.READ_ONLY | cl.mem_flags.COPY_HOST_PTR,
        img_format,
        shape=(temp_w, temp_h),
```

```
        hostbuf=template_minus_avg
    )

    # Allocate device-side images
    imageIn = cl.Image(gm.ctx, cl.mem_flags.READ_ONLY, img_format,
       shape=(img_w, img_h))

    # Output correlation map
    output_buff_shape = [img_h - temp_h + 1 ,  img_w - temp_w + 1]
    output_buff = np.zeros([output_buff_shape[0], output_buff_shape
       [1]], dtype=np.uint8)
    imageOut = cl.create_image(
        gm.ctx,
        cl.mem_flags.WRITE_ONLY | cl.mem_flags.COPY_HOST_PTR,
        img_format_uint8,
        shape = (output_buff_shape[1], output_buff_shape[0]),
        hostbuf = output_buff
    )

    kernel = gm.kernel_ncc_tiled

    local_work_size = (8, 8)
    global_work_size = (
        math.ceil((output_buff_shape[1]) / local_work_size[0]) *
           local_work_size[0],
        math.ceil((output_buff_shape[0]) / local_work_size[1]) *
           local_work_size[1]
    )


    input_image = input_image.astype(np.float32)
    # Update device memory for the current sub-image
    cl.enqueue_copy(gm.commQ, imageIn, input_image, origin=(0,0,0),
        region=(img_w, img_h, 1), is_blocking=True)

    # Kernel: Tiled Normal Correlation Coefficient (ncc_tiled)
    kernel.set_arg(0, np.int32(img_w))
    kernel.set_arg(1, np.int32(img_h))
    kernel.set_arg(2, np.float32(template_sum_mean))
    kernel.set_arg(3, imageIn)
    kernel.set_arg(4, imageIn_template)
    kernel.set_arg(5, imageOut)

    cl.enqueue_nd_range_kernel(gm.commQ, kernel, global_work_size,
       local_work_size)
```

```
    cl.enqueue_copy(gm.commQ, output_buff, imageOut, origin=(0,0,0)
       , region=(output_buff_shape[1], output_buff_shape[0], 1))
    gm.commQ.finish()

    if verbose:
        print(output_buff)
        cv.imshow("frame", output_buff)
        cv.waitKey(0)

    return output_buff

def linear_transform_img(gm: GameManager, input_image: np.array,
   img_shift: np.array, verbose: bool = False):
    img_h, img_w = input_image.shape[:2]
    input_image = cv.cvtColor(input_image, cv.COLOR_BGR2RGBA)

    try:
        img_format_uint = cl.ImageFormat(cl.channel_order.RGBA, cl.
           channel_type.UNSIGNED_INT8)

        # Create OpenCL images
        imageIn = cl.Image(gm.ctx, cl.mem_flags.READ_ONLY | cl.
           mem_flags.COPY_HOST_PTR,
                           img_format_uint, shape=(img_w, img_h),
                           hostbuf=input_image)

        imageOut = cl.Image(gm.ctx, cl.mem_flags.WRITE_ONLY,
           img_format_uint, shape=(img_w, img_h))


        # Set kernel arguments
        kernel = gm.kernel_linear_transform_img
        kernel.set_arg(0, np.int32(img_w))
        kernel.set_arg(1, np.int32(img_h))
        kernel.set_arg(2, np.int32(img_shift[0]))
        kernel.set_arg(3, np.int32(img_shift[1]))
        kernel.set_arg(4, imageIn)
        kernel.set_arg(5, imageOut)

        # Enqueue kernel execution
        cl.enqueue_nd_range_kernel(gm.commQ, kernel, (img_w, img_h)
           , None)
        gm.commQ.finish()
```

```
        # Allocate host array to copy result
        output = np.zeros((img_h, img_w, 4), dtype=np.uint8)
        cl.enqueue_copy(gm.commQ, output, imageOut, origin=(0, 0,
           0), region=(img_w, img_h, 1))
        gm.commQ.finish()

        output = cv.cvtColor(output, cv.COLOR_RGBA2BGR)

        if verbose:
            plt.imshow(output)
            plt.axis('off')
            plt.show()
        return output

    except Exception as e:
        print("From linear_transform_img error:", e)
```

Listing 1: kernel funcs.py

```
__kernel void power2(__global int* arr)
{
    int i = get_global_id(0);
    if (i < 25){
        arr[i] = arr[i] * arr[i];
    }
}

__kernel void power_const(__global int* arr, int K, __global int* result)
{
    int i = get_global_id(0);
    if (i > 25)
        return;
    arr[i] = arr[i] * K;
    result[0] += arr[i];
}

__kernel void negative(__global uchar* image, int w, int h, int padding,
    __global uchar* imageOut)
{
    int x = get_global_id(0);
    int y = get_global_id(1);
    int idx = y * (w*3 + padding) + x*3 ;
    if ((x < w) && (y < h)) { // check if x and y are valid image coordinates
        imageOut[idx] = 255 - image[idx];
        imageOut[idx+1] = 255 - image[idx+1];
        imageOut[idx+2] = 255 - image[idx+2];
    }
}


__kernel void brightness_and_contrast(int w, int h, int padding, int brightness,
    float contrast, __read_only image2d_t imageIn, __global uchar4* imageOut)
```

```
{
    const sampler_t sampler = CLK_NORMALIZED_COORDS_FALSE |
                              CLK_ADDRESS_CLAMP |
                              CLK_FILTER_NEAREST;

    int x = get_global_id(0);
    int y = get_global_id(1);
    if (x >= w || y >= h) return;

    float4 pixel = read_imagef(imageIn, sampler, (int2)(x, y)) * 255.0f;

    pixel.x = clamp(mad(contrast, pixel.x, brightness), 0.0f, 255.0f);
    pixel.y = clamp(mad(contrast, pixel.y, brightness), 0.0f, 255.0f);
    pixel.z = clamp(mad(contrast, pixel.z, brightness), 0.0f, 255.0f);
    pixel.w = 255.0f;

    int idx = y * w + x;
    imageOut[idx] = (uchar4)(pixel.x, pixel.y, pixel.z, pixel.w);
}

__kernel void sobel(int w, int h,
                    __read_only image2d_t imageIn,
                    __global uchar4* imageOut)
{
    const sampler_t sampler = CLK_NORMALIZED_COORDS_FALSE |
                              CLK_ADDRESS_CLAMP_TO_EDGE |
                              CLK_FILTER_NEAREST;

    int x = get_global_id(0);
    int y = get_global_id(1);
    if (x >= w || y >= h) return;

    // Sobel kernels
    int Gx[3][3] = {{1, 0, -1},
                    {2, 0, -2},
                    {1, 0, -1}};
    int Gy[3][3] = {{-1, -2, -1},
                    { 0,  0,  0},
                    { 1,  2,  1}};

    uint4 gx = 0.0f, gy = 0.0f;

    for (int i = -1; i <= 1; ++i) {
        for (int j = -1; j <= 1; ++j) {
            uint4 p = read_imageui(imageIn, sampler, (int2)(x + j, y + i));
            gx += p * Gx[i+1][j+1];
            gy += p * Gy[i+1][j+1];
        }
    }

    float4 g = sqrt(convert_float4(gx*gx + gy*gy));
    g = clamp(g, 0.0f, 255.0f);
    uint4 g_uint = convert_uint4(g);

    int idx = y * w + x;
```

```
87      imageOut[idx] = (uchar4)(g_uint.x, g_uint.y, g_uint.z, 255);
88  }
89
90
91  __kernel void subtract_val_to_img(
92      int w,
93      int h,
94      float4 val_to_sub,
95      __read_only image2d_t imageIn,
96      __write_only image2d_t imageOut)
97  {
98      const sampler_t sampler =
99          CLK_NORMALIZED_COORDS_FALSE |
100         CLK_ADDRESS_CLAMP_TO_EDGE |
101         CLK_FILTER_NEAREST;
102
103     int x = get_global_id(0);
104     int y = get_global_id(1);
105     if (x >= w || y >= h) return;
106
107     float4 p = read_imagef(imageIn, sampler, (int2)(x, y));
108     write_imagef(imageOut, (int2)(x, y), p - val_to_sub);
109 }
110
111 __kernel void img_mult_pix2pix(
112     int w, int h,
113     __read_only image2d_t imageIn_1,
114     __read_only image2d_t imageIn_2,
115     __write_only image2d_t out_put_mult
116 )
117 {
118     const sampler_t sampler =
119         CLK_NORMALIZED_COORDS_FALSE |
120         CLK_ADDRESS_CLAMP_TO_EDGE |
121         CLK_FILTER_NEAREST;
122
123     int x = get_global_id(0);
124     int y = get_global_id(1);
125
126     if (x >= w || y >= h)
127         return;
128
129     float4 p_1 = read_imagef(imageIn_1, sampler, (int2)(x, y));
130     float4 p_2 = read_imagef(imageIn_2, sampler, (int2)(x, y));
131
132     float4 result = p_1 * p_2;
133
134     write_imagef(out_put_mult, (int2)(x, y), result);
135 }
136
137 #define LOCAL_H 8
138 #define LOCAL_W 8
139 #define TEMP_H 105
140 #define TEMP_W 90
141
```

```
142 __kernel void ncc_tiled(
143     int img_w,
144     int img_h,
145     float template_sum_mean,
146     __read_only image2d_t inputImage,
147     __read_only image2d_t templateImage_min_avg,
148     __write_only image2d_t outputImage)
149 {
150     const sampler_t sampler =
151         CLK_NORMALIZED_COORDS_FALSE |
152         CLK_ADDRESS_CLAMP_TO_EDGE |
153         CLK_FILTER_NEAREST;
154 
155     const int global_x = get_global_id(0);
156     const int global_y = get_global_id(1);
157 
158     const int group_x = get_group_id(0) * LOCAL_W;
159     const int group_y = get_group_id(1) * LOCAL_H;
160 
161     const int local_x = get_local_id(0);
162     const int local_y = get_local_id(1);
163 
164     // Shared tile in local memory
165     __local float tile[LOCAL_H + TEMP_H - 1][LOCAL_W + TEMP_W - 1];
166 
167     /* Load tile */
168     for (int y = local_y; y < LOCAL_H + TEMP_H - 1; y += LOCAL_H) {
169         for (int x = local_x; x < LOCAL_W + TEMP_W - 1; x += LOCAL_W) {
170             int2 coord = (int2)(group_x + x, group_y + y);
171             float4 pix = read_imagef(inputImage, sampler, coord);
172             tile[y][x] = pix.x; // take only the first channel (grayscale)
173         }
174     }
175 
176     barrier(CLK_LOCAL_MEM_FENCE);
177 
178     /* Skip out-of-bounds work-items */
179     if (global_x >= img_w - TEMP_W + 1 || global_y >= img_h - TEMP_H + 1) return
    ;
180 
181     // Compute sub-image average
182     float sub_img_avg = 0.0f;
183     float sub_img_sum_square = 0.0f;
184     for (int template_y = 0; template_y < TEMP_H; ++template_y) {
185         for (int template_x = 0; template_x < TEMP_W; ++template_x) {
186             sub_img_avg += tile[local_y + template_y][local_x + template_x];
187             sub_img_sum_square += tile[local_y + template_y][local_x +
    template_x]*tile[local_y + template_y][local_x + template_x];
188         }
189     }
190     sub_img_avg /= (TEMP_H * TEMP_W);
191 
192     // Compute correlation
193     float correlation = 0.0f;
194     for (int template_y = 0; template_y < TEMP_H; ++template_y) {
```

```
195         for (int template_x = 0; template_x < TEMP_W; ++template_x) {
196             float sub_pix = tile[local_y + template_y][local_x + template_x];
197             float tmp_min_avg = read_imagef(templateImage_min_avg, sampler, (
    int2)(template_x, template_y)).x;
198             correlation += (sub_pix - sub_img_avg) * tmp_min_avg;
199         }
200     }
201 
202     correlation = correlation/sqrt(template_sum_mean * sub_img_sum_square);
203     correlation = (correlation + 1) * 127.5f;
204 
205     write_imageui(outputImage, (int2)(global_x, global_y), (uint4)((uint)
    correlation, 0, 0, 255));
206 }
207 
208 __kernel void linear_transform_img(
209     int img_w,
210     int img_h,
211     int shift_x,
212     int shift_y,
213     __read_only image2d_t inputImage,
214     __write_only image2d_t outputImage)
215 {
216     const sampler_t sampler =
217         CLK_NORMALIZED_COORDS_FALSE |
218         CLK_ADDRESS_CLAMP_TO_EDGE |
219         CLK_FILTER_NEAREST;
220 
221     const int global_x = get_global_id(0);
222     const int global_y = get_global_id(1);
223 
224     if (global_x - shift_x <= 0 || global_y - shift_y <= 0 || global_x -
    shift_x >= img_w || global_y - shift_y >= img_h) return;
225 
226     uint4 pixel = read_imageui(inputImage, sampler, (int2)(global_x - shift_x,
    global_y - shift_y));
227 
228     write_imageui(outputImage, (int2)(global_x, global_y), pixel);
229 }
```

Listing 2: prog.c